

Exploring k-Nearest Neighbors in Recommendation Systems

Daizy Buluma*

Department of Statistics, Amherst College

December 14, 2025

Abstract

Recommender systems help users navigate vast digital catalogs by predicting preferences. This paper examines k-Nearest Neighbors (kNN) as a baseline collaborative filtering method, contrasting it with matrix factorization via Singular Value Decomposition (SVD). Using the Book-Crossing dataset, preprocessing addressed sparsity and produced a denser ratings matrix for evaluation. Accuracy was assessed with RMSE, MAE and MSE across neighborhood sizes and similarity measures. Results show kNN offers transparency and pedagogical value, while SVD achieves superior predictive accuracy. An interactive Shiny app demonstrates kNN in practice.

Keywords: Collaborative filtering, Similarity metrics, Cold start problem, Matrix factorization, Performance evaluation, Book-Crossing dataset

*The author gratefully acknowledges Vanesa for her thoughtful peer review and constructive feedback. Sincere thanks are also extended to the author's family for their unwavering support and encouragement throughout the process.

1 Introduction

Today’s digital world is overflowing with information, making it nearly impossible for individuals to manually sift through the massive number of available options. As a result, recommender systems have become essential components of platforms such as Netflix, Amazon, Spotify, and TikTok. Their primary goal is to filter vast collections of items such as movies, products, songs, or videos, and present users with options they are most likely to enjoy. These systems work by analyzing users’ preferences, past behaviors, and item characteristics through interaction data such as impressions, clicks, ratings, likes, and purchases (Ghosh 2024).

This paper focuses on the k-nearest neighbors (kNN) algorithm within the context of recommendation systems. Although kNN is one of the more straightforward algorithms in machine learning, it plays a foundational role in collaborative filtering methods. kNN identifies users or items that are most similar in behavior or preferences and bases predictions on these neighbors. Its simplicity and intuitive mechanics make it especially suitable for educational purposes and small scale recommendation tasks.

Before exploring the details of kNN, this paper outlines the broader landscape of recommendation systems to provide a foundation for understanding how kNN fits into the larger family of recommendation systems. The discussion then introduces and contrasts a different recommendation approach, Singular Value Decomposition (SVD) to highlight the strengths and weaknesses of kNN in practice.

Finally, the second half of the paper applies kNN to a real dataset to build and evaluate a simple interactive recommender system so as demonstrate how the method works in practice and gauge its performance.

2 Overview of Recommender Systems

Recommendation systems work by estimating how much a user is likely to enjoy an item they have not yet interacted with. They do this by analyzing **user behavior**, such as past ratings or interactions and **item features**, such as genre, creator, or textual descriptions. The goal is to estimate a preference score, $r_{u,i}$ which measures how likely user u is to enjoy item i . Recommendations are then generated by selecting the items with the highest scores. More details in section 3.3 below.

To understand how k-nearest neighbors (kNN) fits into this broader landscape, we will discuss two major families of recommendation approaches: content-based approach and collaborative filtering (There are hybrid recommenders too which combine the two approaches).

2.1 Content-Based Filtering

Content-based recommender systems generate suggestions by comparing the attributes of items a user previously liked to those of new items. These attributes might include genres, keywords, textual descriptions, or metadata. The item descriptions, which are labeled with ratings, are used as training data to create a user-specific classification or regression modeling problem. As Aggarwal explains, for each user the training documents correspond to the descriptions of the items she has bought or rated (Aggarwal 2016, p. 14). For example, if a shopper frequently purchases skincare products, the system recommends related cosmetic items.

The underlying assumption is that user preferences are largely consistent, and similar items will yield similar satisfaction (Ghosh 2024). These models rely heavily on item features,

making them effective in domains where rich metadata is available.

2.2 Collaborative Filtering

Collaborative filtering takes a different approach by focusing on patterns of behavior across users rather than item attributes. The key assumption is that users who behaved similarly in the past will behave similarly in the future. The main challenge in designing collaborative filtering methods is that the underlying ratings matrices are sparse (Most ratings are unspecified). However, in collaborative filtering methods, these unspecified ratings can be imputed because the observed ratings are often highly correlated across various users and items. There are two main subtypes:

2.2.1 Memory-Based Methods

Memory-based methods make predictions directly from observed ratings by identifying similarities among users or items. kNN is the most common example and can be further categorized into user-based kNN and item-based kNN as we'll see in the next sections. The advantages of memory-based techniques are that they are simple to implement and the resulting recommendations are often easy to explain. On the other hand, memory-based algorithms do not work very well with sparse ratings matrices ([Aggarwal 2016](#)).

2.2.2 Model-Based Methods

Model-based approaches use machine learning techniques to uncover latent patterns in the rating matrix. Examples include Matrix factorization, SVD and Neural networks. They tend to outperform memory-based methods on large, sparse datasets but are less

transparent to users and developers. Notably, many hybrid recommenders combine kNN with model-based techniques, such as SVD, to balance simplicity and predictive power ([Hssina et al. 2021](#)).

3 K-Nearest Neighbors in Recommendation Systems

This section provides a deeper exposition of how kNN works, beginning with motivation and intuition, then moving into supervised-learning kNN, and finally examining how the algorithm is adapted for recommendation systems. A worked toy example concludes the section to illustrate the computations concretely.

3.1 Motivation: Why Neighbors Matter in Recommendations

The appeal of neighborhood-based methods lies in their simplicity as people with similar tastes tend to enjoy similar things. If two users have consistently rated the same movies in comparable ways, then the films one user has enjoyed but the other has not yet seen become strong candidates for recommendation. This intuition, that preferences cluster and can be shared across neighbors, forms the foundation of k-Nearest Neighbors (kNN) in recommender systems. By exploiting these local patterns, kNN can generate useful predictions without the need for complex modeling ([Nikolakopoulos et al. 2021](#)).

3.2 How kNN Works in Supervised Learning

Before adapting kNN to recommender systems, it helps to first understand how the algorithm works in its original setting. Unlike most algorithms we've seen in class that estimate

parameters or fit equations, kNN is a “lazy learner”(GeeksforGeeks 2024b). It stores the training data and defers computation until a query arrives. When a new example is presented, the algorithm measures its similarity to all stored examples, selects the k closest ones, and bases its prediction on their values.

In classification tasks, the new example is assigned to the majority class among its neighbors. In regression, the prediction is the average of their values. Weighted variants of kNN give greater influence to closer neighbors, which reduces the impact of outliers. The choice of k is critical to prevent over or under fitting. Too small a neighborhood makes the algorithm sensitive to noise, while too large a neighborhood can dilute meaningful signals.

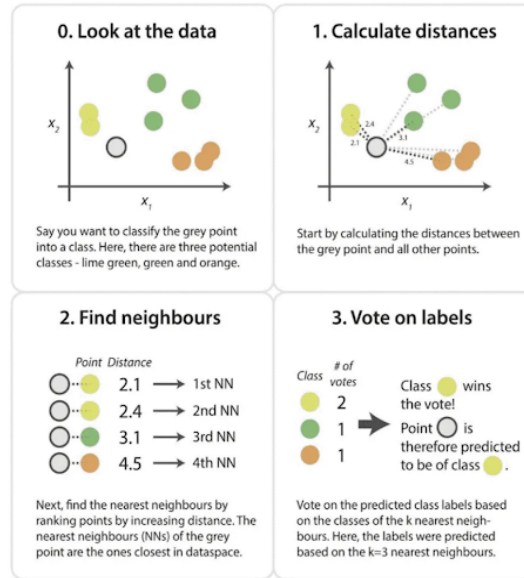


Figure 1: A simple illustration of kNN(source: Sharma 2020)

Because kNN does not assume any particular distribution of the data, it is considered non-parametric. This flexibility makes it easy to apply, but it also introduces challenges in high-dimensional spaces where distances lose contrast, a phenomenon known as the curse of dimensionality (Kramer 2013, pg 19). The curse of dimensionality means that as the number of features grows, data points become increasingly sparse and distance measures

lose meaning, making algorithms like kNN less effective([GeeksforGeeks 2024a](#)).

3.3 How kNN Works in Recommender Systems

In recommendation settings, the task is to predict missing entries in a sparse user-item matrix of size (number of users) $\times$ (number of items). Each row represents a user, each column an item, and most cells are empty because users interact with only a small fraction of available items. Below is an example of such matrix:

User / Item	A	B	C	D
U1	4	4	?	3
U2	4	5	2	?
U3	?	4	4	3
U4	2	?	5	?

Since most cells are empty, kNN adapts by computing similarities only on overlapping ratings. This reliance on shared ratings is both a strength because it uses actual behavioral similarity and a weakness because sparsity may cause the similarities to be unreliable ([Kramer 2013](#)). Two main variants are used:

- **User-based kNN:** The system identifies the k users most similar to the target user and infers preferences from their ratings. For example, if User 1 and User 2 have rated many of the same items similarly, then User 2's rating of a new item can help predict User 1's interest.
- **Item-based kNN:** Instead of comparing users, the system finds items that consistently receive similar ratings across users. If Item A and Item B are rated alike

by many users, then a user who liked Item A is predicted to enjoy Item B as well. Item-based kNN is often more stable because item relationships remain consistent even as new users enter or leave the system. For this reason, modern commercial recommenders tend to use item-based kNN over user-based (Ricci et al. 2022).

Similarity metrics are central to these computations. Euclidean distance provides the most straightforward measure of closeness between rating vectors, but it is sensitive to differences in rating scales and magnitudes. To address this, cosine similarity measures the angle between rating vectors, making it resistant to scale differences. Pearson correlation further adjusts for user biases by centering ratings around individual means. Adjusted cosine similarity, commonly used in item-based kNN, subtracts user averages before computing similarity to reduce the effect of rating habits (Aggarwal 2016). In this paper, we use **cosine similarity**, defined as:

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

where $A \cdot B$ is the dot product of the vectors and $\|A\|$, $\|B\|$ are their magnitudes.

The similarity scores typically range from -1 to 1, with -1 for vectors that are polar opposites, 0 where there is no correlation, and 1 for perfectly similar vectors (APXML 2025).

Once similarities are computed, predictions are made using weighted averages. In user-based approach, a simple average of is often misleading because different users have different rating scales. One user might rate movies from 3 to 5 stars, while another uses the full 1-to-5 range. To account for this, we use the deviation from each user’s average rating. For user-based kNN, defines one formula for the predicted rating of user (u) for item (i) as:

$$P_{u,i} = \bar{r}_u + \frac{\sum_{v \in N} \text{sim}(u, v) \cdot (r_{v,i} - \bar{r}_v)}{\sum_{v \in N} |\text{sim}(u, v)|}$$

Where;

- $P_{u,i}$ is the predicted rating for the target user (u) on item (i).
- $\bar{r}_u$ is user (u)'s average rating. We add it at the end so that the prediction is returned to the user's original rating scale.
- N is the neighborhood of users most similar to (u) who have rated item (i).
- $\text{sim}(u, v)$ is the similarity score between the target user (u) and a neighbor (v).
- $r_{v,i} - \bar{r}_v$ is neighbor (v)'s rating of item (i), centered by subtracting their own average rating. This expresses how much higher or lower than average the neighbor rated that item.
- The denominator, $\sum_{v \in N} |\text{sim}(u, v)|$, normalizes the weighted sum so that the prediction isn't inflated by the number or magnitude of similarity scores.

For item-based collaborative filtering, the logic mirrors the user-based approach but is slightly simpler. To predict user (u)'s rating for item (i), we look at the other items that user (u) has already rated. The prediction is formed by taking a weighted average of those ratings, where the weights come from the similarity between item (i) and each neighboring item.

The formula is:

$$P_{u,i} = \frac{\sum_{j \in N} \text{sim}(i, j) \cdot r_{u,j}}{\sum_{j \in N} |\text{sim}(i, j)|}$$

Where;

- $P_{u,i}$ is the predicted rating for user (u) on item (i).
- N is the set of items most similar to item (i) that user (u) has already rated.
- $\text{sim}(i, j)$ is the similarity between the target item (i) and a neighbor item (j).
- $r_{u,j}$ is the rating user (u) gave to item (j).

One important detail is that we do not need to adjust for user averages here. Because all ratings involved in the calculation come from the same user, their rating scale is already consistent across items. As a result, the weighted average directly reflects how much the user liked similar items, making the item-based formulation slightly simpler than the user-based version.

Both formulas and breakdowns were sourced from ([APXML 2025](#)).

A critical design choice in kNN is the size of the neighborhood, denoted by k . The number of neighbors directly influences the bias-variance trade-off in predictions ([for Managers 2025](#)). With very small k , the recommender becomes highly sensitive to noise in individual ratings, which can reduce accuracy. With very large k , the recommender averages across many dissimilar users or items, diluting meaningful signals. In practice, the optimal k is determined empirically by evaluating accuracy metrics such as RMSE, MAE, Precision, and Recall across different values of k ([Aggarwal 2016](#)). Cross-validation is commonly used to select k , ensuring that the chosen neighborhood size generalizes well to unseen data ([Deb 2025](#)).

3.4 Performance Metrics in Recommender Systems

Evaluating recommender systems requires a suite of complementary metrics, since no single measure can fully capture performance. The choice of neighborhood size k in kNN is closely tied to these metrics, as it shapes both prediction accuracy and recommendation quality (Hssina et al. 2021).

- **Prediction Accuracy (ratings-based):**

This answers the question whether the system predicts ratings correctly. Metrics such as **RMSE (Root Mean Squared Error)** and **MAE (Mean Absolute Error)** are used for this purpose. RMSE penalizes large errors more heavily, making it sensitive to extreme mispredictions, while MAE provides an interpretable measure of the average deviation between predicted and actual ratings. Smaller neighborhoods may reduce RMSE when neighbors are highly similar, but they risk instability; larger neighborhoods smooth predictions but can inflate MAE by averaging across dissimilar users (Aggarwal 2016).

- **Recommendation Accuracy (top-N lists):**

Beyond rating prediction, recommender systems must generate useful item suggestions. Here, metrics such as **Precision**, **Recall**, and the **F1 Score** are central. Precision measures the proportion of recommended items that are truly relevant, recall captures the proportion of relevant items successfully recommended, and F1 balances the two. The choice of k directly affects these outcomes: a small neighborhood may yield highly precise but narrow recommendations, while a larger neighborhood increases recall but risks introducing irrelevant items (Ricci et al. 2022).

- **Ranking Quality:**

Finally, recommendation quality depends not only on which items are suggested but also on how they are ordered. Measures such as **ROC curves and AUC** visualize the trade-off between true positives and false positives, while **Precision–Recall curves** highlight performance in sparse datasets where relevant items are rare. These ranking-oriented metrics are particularly important in large catalogs, where most items are irrelevant to any given user ([Hahsler 2022](#)).

3.5 Toy Example

We finish with a toy example adapted from Aggarwal’s book to emphasize idea. Consider a simple dataset of three users and three movies:

User	A	B	C
U1	5	4	?
U2	4	4	2
U3	1	1	1

Entries: Ratings on a 1–5 scale. The “?” is the missing rating we want to predict (U1’s rating for Movie C).

First, we compute the similarity between Movie C and movies that U1 has already rated (A and B) using cosine similarity.

- **Movie A ratings vector:** [5, 4, 1] from U1, U2, U3
- **Movie B ratings vector:** [4, 4, 1] from U1, U2, U3

- **Movie C ratings vector:** $[?, 2, 1] \rightarrow$ U1's rating is missing, so we only use U2 and U3: $[2, 1]$

Now,

To compute $\mathbf{sim}(\mathbf{A}, \mathbf{C})$, we compare Movie A's ratings from U2 and U3 ($[4, 1]$) with Movie C's ratings from U2 and U3 ($[2, 1]$).

$$\mathit{sim}(A, C) = \frac{(4 \cdot 2) + (1 \cdot 1)}{\sqrt{4^2 + 1^2} \cdot \sqrt{2^2 + 1^2}} \approx 0.98$$

To compute $\mathbf{sim}(\mathbf{B}, \mathbf{C})$, we compare Movie B's ratings from U2 and U3 ($[4, 1]$) with Movie C's ratings from U2 and U3 ($[2, 1]$).

$$\mathit{sim}(B, C) = \frac{(4 \cdot 2) + (1 \cdot 1)}{\sqrt{4^2 + 1^2} \cdot \sqrt{2^2 + 1^2}} \approx 0.96$$

These values (0.98 and 0.96) show that Movies A and B are both highly similar to Movie C therefore we use their ratings to calculate the predicted rating for Movie C.

Plugging in values into the Item-Based formula we get:

$$P_{U1,C} = \frac{(0.98 \cdot 5) + (0.96 \cdot 4)}{0.98 + 0.96} = \frac{4.9 + 3.84}{1.94} \approx 4.55$$

U1's predicted rating for Movie C is about 4.6 stars. The recommendation system expects U1 to enjoy Movie C almost as much as Movies A and B.

3.6 Strengths and Weaknesses of kNN in Recommendation Systems

As we established in the sections above, kNN requires no training phase, is easy to explain, and works well for small or dense datasets where overlaps are plentiful. Its neighbor-driven reasoning makes it particularly suitable for educational settings and for systems where interpretability is valued.

However, kNN also has notable weaknesses. Its reliance on overlapping ratings makes it vulnerable to sparsity, and its computational cost grows quickly with dataset size. In high-dimensional rating matrices, distance metrics can become unreliable therefore reducing accuracy. These limitations explain why kNN is often used as a baseline method, while large-scale commercial systems like Net-flix or Spotify, where accuracy and scalability matter most rely on more advanced model-based approaches such as matrix factorization or deep learning ([Hssina et al. 2021](#)).

4 Comparison to a Model-Based Method (SVD)

While kNN provides an intuitive way to generate recommendations by looking at neighbors, it struggles with sparse data and scalability([Aggarwal 2016](#), [Ricci et al. 2022](#)). To address these challenges, recommender systems often turn to **model-based methods**. One of the most influential is **Singular Value Decomposition (SVD)**, a matrix factorization technique that has become central to modern recommendation engines.

4.1 What SVD Does (Very High-Level)

SVD takes the large (millions of ratings), sparse user-item ratings matrix (R) and decomposes it into three smaller matrices([APXML 2025](#)):

$$R_{m \times n} = U_{m \times r} \cdot \Sigma_{r \times r} \cdot V_{r \times n}^T$$

- $U_{m \times r}$: Represents the m users in terms of hidden “taste dimensions” (latent factors). Each row corresponds to a user’s coordinates in the reduced latent space.
- $\Sigma_{r \times r}$: Diagonal matrix of singular values, showing which factors are most important.
- $V_{r \times n}^T$: Represents the n items in terms of the same latent factors. Each column corresponds to an item’s coordinates in the latent space.

By keeping only the top k singular values, SVD compresses the ratings matrix into a lower-dimensional representation([APXML 2025](#)). This reveals hidden patterns of preference for example, one factor might capture a user’s tendency toward action/sci-fi movies, while another captures romance/musicals. Predictions for missing ratings are made by combining a user’s latent profile with an item’s latent profile ([APXML 2025](#)).

This approach differs fundamentally from kNN. While kNN Predictions rely on local neighborhoods and finds similar users or items directly from observed ratings, SVD learns a compressed representation of the entire dataset and its predictions come from latent factors rather than direct neighbors.

kNN is transparent and easy to interpret (“your neighbor liked this, so you might too”), but it struggles with sparsity and scalability([Ricci et al. 2022](#)). SVD, by contrast, is less interpretable because latent factors are abstract, yet it generalizes better and scales efficiently to millions of users and items.

The trade-offs between the two methods can be summarized as follows:

Aspect	kNN (Memory-Based)	SVD (Model-Based)
Mechanism	Direct similarity between users/items	Factorizes ratings matrix: $R = U\Sigma V^T$
Data requirement	Works directly on raw ratings	Needs enough data to learn latent factors
Scalability	Struggles with millions of users/items	Efficient at large scale once trained
Cold start problem	Severe (new users/items have no neighbors)	Still an issue, but generalizes better
Accuracy	Good for small datasets, local patterns	Better for large datasets, global patterns
Interpretability	Easy to explain (“neighbors liked it”)	Harder to explain (latent factors abstract)

The cold start problem arises when new users or items have too few ratings to support reliable recommendations. In kNN this issue is severe, since similarity requires overlapping ratings; SVD also faces cold start challenges but can generalize better once more data is available ([Aggarwal 2016](#)).

5 Application: Building a Simple Recommendation Engine with kNN

This analysis uses the Book-Crossing dataset, originally compiled by Cai-Nicolas Ziegler and later made available on Kaggle (Ziegler 2024). The implementation relies on the R package recommenderlab (Hahsler 2025), which provides standardized data structures (realRatingMatrix) and functions for collaborative filtering, evaluation schemes, and model comparison. Code snippets for model training and evaluation were adapted from the official recommenderlab documentation (Hahsler 2025). The dataset consists of three interconnected tables:

- **Books dataset:** Contains metadata for each book, including ISBN, title, and author.
- **Ratings dataset:** Records user interactions with books, where each entry includes a `user_id`, `book_id`, and a numerical rating on a 0–10 scale.
- **Users dataset:** Provides demographic information for each user, including `user_id`, age, and location.

A snippet of these datasets can be found in section 6.1 of the appendix.

5.1 Data Preprocessing

We begin by loading the three source files and cleaning them to retain only the relevant attributes. For books, we keep the ISBN, title, and author; for ratings, we rename the columns to `user_id`, `book_id`, and `book_rating`; and for users, we retain the `user_id` and `location`. There were no duplicate entries or missing values.

```

books <- books |>

  select(ISBN,Book.Title,Book.Author) |>

  rename(

    book_title = Book.Title,

    author = Book.Author,

    book_id = ISBN

  )

ratings <- ratings |>

  rename(

    user_id = User.ID,

    book_rating = Book.Rating,

    book_id = ISBN

  )

users <- users |>

  select(User.ID, Location,Age) |>

  rename(

    user_id = User.ID,

    location = Location,

    age = Age)

```

The three datasets are then joined into a single ratings table, providing a unified view of user–book interactions.

```

ratings_full <- ratings |>
  inner_join(books,
             by = "book_id")|>
  inner_join(users,
             by = "user_id")

```

After joining the three datasets, the combined table contains over 1 million ratings from approximately 92,000 users on nearly 270,000 books.

This raw dataset is highly sparse which means most users rate only a handful of books, and most books receive very few ratings. Such sparsity is typical in recommendation systems and motivates the filtering and preprocessing steps described below.

Ratings in the full dataset range from 0 to 10. However summary statistics revealed that more than half of the ratings in the dataset were equal to zero.

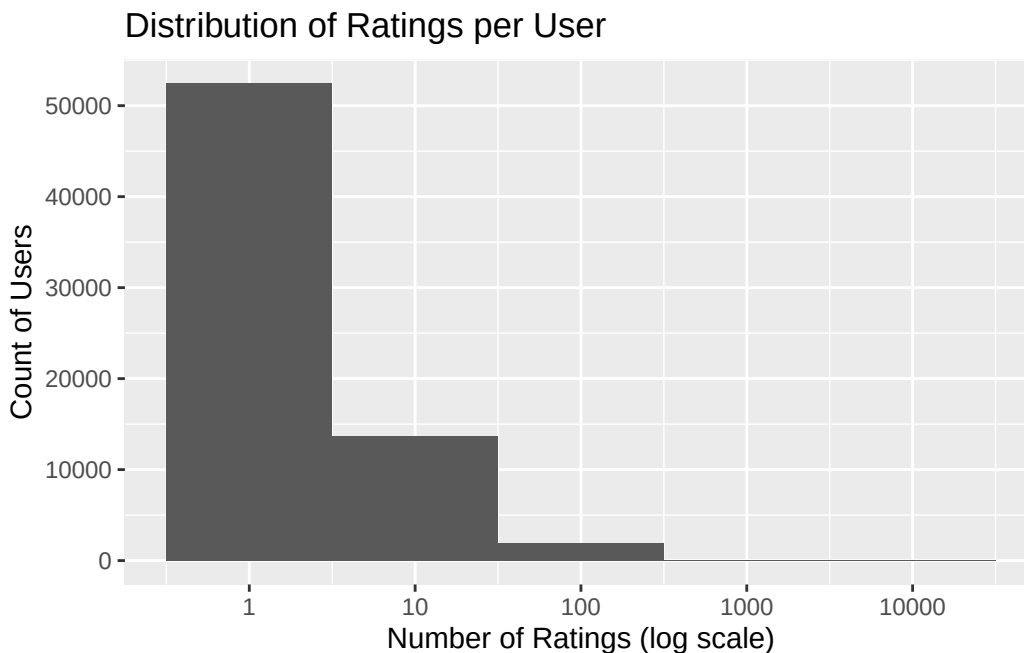
Table 4: Distribution of Ratings in the Book-Crossing Dataset

0	1	2	3	4	5	6	7	8	9	10
647294	1481	2375	5118	7617	45355	31687	66402	91804	60778	71225

The zero values were interpreted as indicators that a user had interacted with a book without providing explicit feedback and not as genuine ratings. In other words, a 0 denotes presence in the system rather than an expression of preference. Since collaborative filtering depends on explicit numerical ratings to compute similarities and generate predictions (Aggarwal 2016), including these implicit entries would skew the rating distribution and distort accuracy measures. To avoid this bias, all records with a rating of zero were excluded from the analysis.

```
# Drop records with rating = 0  
ratings_clean <- ratings_full |>  
  filter(book_rating > 0)
```

Because the raw dataset is sparse, we only include users and books with sufficiently active profiles.



The histogram above shows the distribution of how many books each user has rated in the dataset. Because the x-axis is plotted on a log scale, we can see that the majority of users contributed only a small number of ratings, while a much smaller group of highly active users contributed thousands. This is because most users interact with only a handful of items, whereas a few users generate a disproportionately large number of ratings. This motivates the decision to set a threshold for “active” users to ensure that the recommender system is trained on profiles with enough ratings to support meaningful recommendations.

This same pattern was found with the items with most books being rated only once or twice, while a small number of popular titles received hundreds of ratings (See histogram

in appendix 6.2). To mitigate this, we introduced a threshold line at 10 ratings, excluding books with fewer than 10 ratings from the final dataset. This cutoff ensures that the recommender system is trained on items with sufficient feedback to support meaningful recommendations.

```
# Set thresholds

min_user_ratings <- 10 # active users
min_item_ratings <- 10 # active books

# Filter ratings dataset to keep only active users and active books
ratings_filtered <- ratings_clean |>

  inner_join(user_counts |> filter(n_user >= min_user_ratings),
             by = "user_id") |>

  inner_join(item_counts |> filter(n_item >= min_item_ratings),
             by = "book_id")
```

The filtering process substantially reduced the dataset, as shown in table below.

Table 5: Dataset size before and after filtering thresholds

Stage	Users	Books	Ratings
Before filtering	68091	149836	383842
After filtering	6318	5438	87178

Before applying thresholds, the dataset contained 68,091 users, 149,836 books and 383,842 ratings. After requiring each user and each book to have at least 10 ratings, the dataset was reduced to 6,318 users, 5,438 books, and 87,178 ratings. This reduction highlights

the sparsity of the original dataset, where most users and books had very few ratings. By retaining only active users and active books, the final dataset provides a denser interaction matrix that supports more reliable similarity calculations and improves the robustness of the recommender system.

5.2 Matrix Transformation

Collaborative filtering requires a user-item matrix. The filtered ratings are reshaped into wide format, converted to a matrix, and then cast into a `realRatingMatrix` object, the format expected by `recommenderlab` ([Hahsler 2025](#))

```
# reshape into wide format (users as rows, books as columns)
rating_wide <- ratings_filtered |>
  select(user_id, book_id, book_rating) |>
  pivot_wider(names_from = book_id, values_from = book_rating)
# convert to matrix (drop user_id column)
rating_matrix <- as.matrix(rating_wide[, -1])
# convert to realRatingMatrix
rating_matrix <- as(rating_matrix, "realRatingMatrix")
```

5.3 Implementing kNN & Model Evaluation

Once the ratings matrix was transformed into a `realRatingMatrix`, the next step was to implement a k-Nearest Neighbors recommender. In `recommenderlab`, this is achieved with the `Recommender()` function, specifying “UBCF” (user-based collaborative filtering) or “IBCF” (item-based collaborative filtering).

To evaluate performance, two complementary schemes can be used. The code snippet below illustrates a simple train/test split. The model is trained on 80% of the data, predictions are generated for the “known” portion of the test set, and accuracy is measured against the “unknown” ratings.

```
# Set a random seed for reproducibility
set.seed(495)

# Define an evaluation scheme:
# method = "split": simple train/test split
# train = 0.8: use 80% of the data for training
# given = 10: assume 10 ratings per user are known in the test set
# goodRating = 6: ratings >= 6 are considered relevant items

scheme_split <- evaluationScheme(rating_matrix,
                                method="split",
                                train=0.8,
                                given=10,
                                goodRating=6)

# Train UBCF on training set
# nn = neighborhood size
# method = similarity measure
# normalize center ratings before similarity calculation

ubcf_model <- Recommender(getData(scheme_split, "train"),
                          method="UBCF",
                          parameter=list(
                            nn=30, method="Cosine", normalize="center"))
```

```

# Generate predicted ratings for the "known" portion of the test set
ubcf_pred <- predict(ubcf_model, getData(scheme_split, "known"),
                    type="ratings")

# Evaluate prediction accuracy against the "unknown" ratings:

# This computes metrics such as RMSE, MSE, and MAE
metrics <- calcPredictionAccuracy(ubcf_pred, getData(scheme_split,
                                                    "unknown"))

```

Because a single split can be sensitive to how the data is partitioned, the analysis relied primarily on k-fold cross-validation for robust model selection. In this scheme, the dataset is divided into five folds; each fold is used once as a test set while the remaining folds serve as training data. Results are averaged across folds, providing a more reliable estimate of performance. Cross-validation was used to compare different neighborhood sizes (k) and similarity measures (Cosine vs. Pearson), ensuring that the chosen configuration generalizes well to unseen data ([Aggarwal 2016](#), [Ricci et al. 2022](#)).

```

# 5-fold cross-validation for exploratory runs
set.seed(495)

scheme_cv_light <- evaluationScheme(rating_matrix,
                                    method="cross-validation",
                                    k=5,
                                    given=10,
                                    goodRating=6)

# Compare two algorithms

```



```

algorithms_grid_light <- list(

  "UBCF_cosine_k10" = list(name="UBCF",

                           param=list(nn=10, method="Cosine",

                                       normalize="center")),

  "UBCF_pearson_k30" = list(name="UBCF",

                             param=list(nn=30, method="Pearson",

                                         normalize="center"))

)

# Ratings-based CV

set.seed(495)

cv_results_ratings_light <- evaluate(scheme_cv_light,

                                    algorithms_grid_light,

                                    type="ratings")

# Summarize RMSE/MAE

ratings_summary_light <- bind_rows(lapply(cv_results_ratings_light, avg),

                                   .id="model")

```

Based on the exploratory cross-validation results (See appendix 6.3), user-based collaborative filtering with cosine similarity and a neighborhood size of $k = 10$ was selected as the final model. To quantify its predictive accuracy, the model was evaluated using a train/test split. This provides an estimate of how well the chosen configuration generalizes to unseen data.

```

# Evaluation scheme for reproducibility (train/test split)

set.seed(495)

scheme_final <- evaluationScheme(rating_matrix,

                                method="split",

                                train=0.8,

                                given=10,

                                goodRating=6)

# Train final model on training set

final_model_eval <- Recommender(

  getData(scheme_final, "train"),

  method      = "UBCF",

  parameter = list(nn = 10, method = "Cosine", normalize = "center")

)

# ---- Ratings-based evaluation ----

final_pred_ratings <- predict(

  final_model_eval,

  getData(scheme_final, "known"),

  type="ratings")

metrics_ratings <- calcPredictionAccuracy(

  final_pred_ratings,

  getData(scheme_final, "unknown"))

```

The final model (UBCF with cosine similarity, $k = 10$) achieved an RMSE of 1.89, MSE of 3.57, and MAE of 1.41 on the train/test evaluation scheme. These values indicate that

predictions deviate from true ratings by approximately 1–2 points on average, which is consistent with performance reported in similar collaborative filtering studies. To contextualize these results, we compared the chosen kNN approach against a baseline matrix factorization method (SVD). This comparison highlights the relative strengths of neighborhood-based collaborative filtering in capturing user preferences within the dataset.

Table 6: Performance comparison of UBCF and SVD models

Model	RMSE	MSE	MAE
UBCF (Cosine, k=10)	1.89	3.57	1.41
SVD (k=50)	1.60	2.56	1.19

As can be seen in the table above SVD achieved lower error values across all three metrics. These results confirm the broader finding in recommender system literature that matrix factorization methods such as SVD typically outperform neighborhood-based approaches in terms of predictive accuracy. Nevertheless, the kNN framework remains valuable for pedagogical purposes, as its mechanics are more transparent and intuitive.

5.4 Interactive Shiny Application

To illustrate how recommendation works in practice, we developed an interactive Shiny application that implements the final User-Based Collaborative Filtering (UBCF) model with cosine similarity. The app provides a hands-on interface where users can select a specific user ID, adjust the neighborhood size k and specify the number of recommendations to generate. The output is a ranked table of book titles and authors, accompanied by preference scores that reflect the relative strength of each recommendation. These scores

are not bounded by the original 1–10 rating scale but serve as indicators of predicted interest, with higher values corresponding to stronger recommendations.

The application also includes a User Lookup tab, which retrieves demographic information such as age and location from the users file. This feature allows researchers to contextualize recommendation patterns by examining how user characteristics relate to the books suggested. Screenshots of both tabs are included below to demonstrate the workflow: one showing the recommendation interface with sliders and output table, and another highlighting the user lookup functionality.

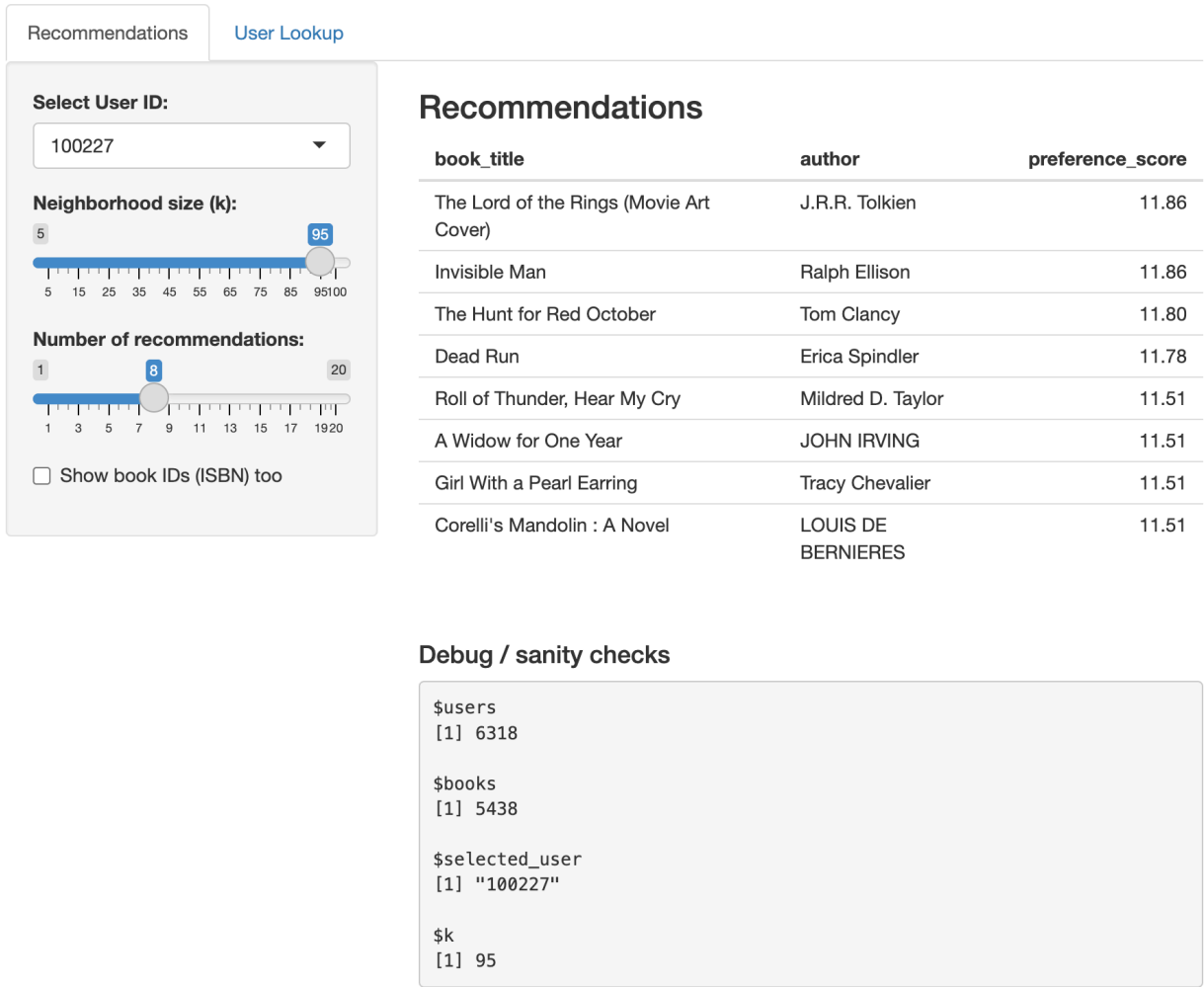


Figure 5 illustrates the Recommendations tab, where users can adjust the neighborhood size and number of recommendations.

[Recommendations](#)

User Lookup

Enter User ID:

100227

Lookup

User Information

user_id	Location	Age
100227	tehran, tehran, iran	36.00

Figure 2: user lookup interface

6 Appendix

6.1 Dataset Samples

Table 7: Sample of Books Dataset

book_id	book_title	author
0195153448	Classical Mythology	Mark P. O. Morford
0002005018	Clara Callan	Richard Bruce Wright
0060973129	Decision in Normandy	Carlo D’Este
0374157065	Flu: The Story of the Great Influenza Pandemic of 1918 and the Search for the Virus That Caused It	Gina Bari Kolata
0393045218	The Mummies of Urumchi	E. J. W. Barber
0399135782	The Kitchen God’s Wife	Amy Tan
0425176428	What If?: The World’s Foremost Military Historians Imagine What Might Have Been	Robert Cowley
0671870432	PLEADING GUILTY	Scott Turow
0679425608	Under the Black Flag: The Romance and the Reality of Life Among the Pirates	David Cordingly
074322678X	Where You’ll Find Me: And Other Stories	Ann Beattie

Table 8: Sample of Ratings Dataset

user_id	book_id	book_rating
276725	034545104X	0
276726	0155061224	5
276727	0446520802	0
276729	052165615X	3
276729	0521795028	6
276733	2080674722	0
276736	3257224281	8
276737	0600570967	6
276744	038550120X	7
276745	342310538	10

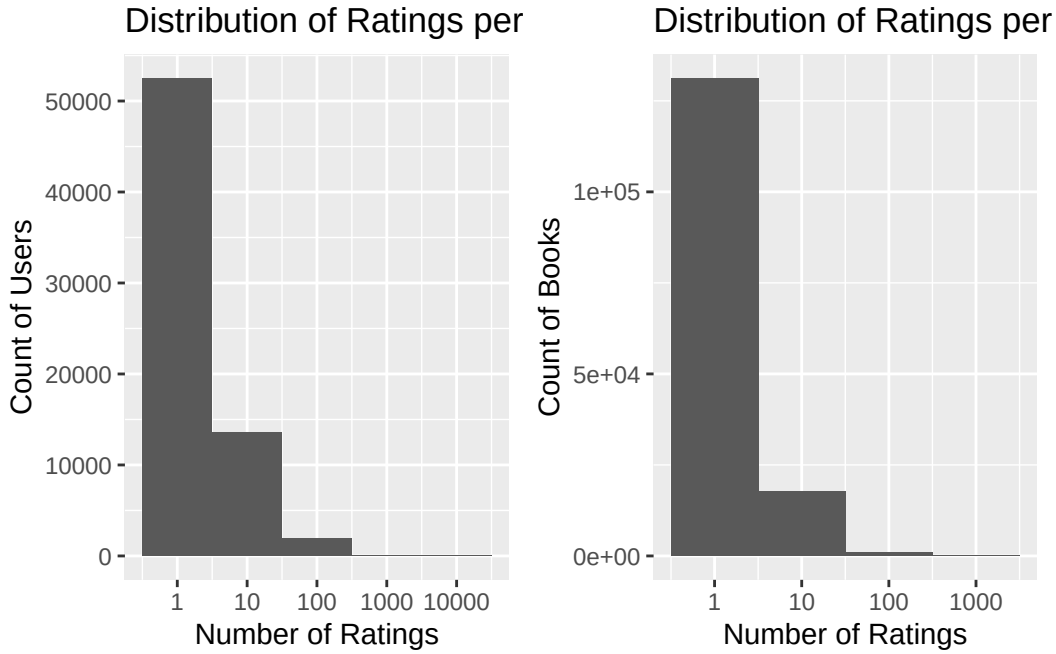
Table 9: Sample of Users Dataset

user_id	location	age
1	nyc, new york, usa	NA
2	stockton, california, usa	18
3	moscow, yukon territory, russia	NA
4	porto, v.n.gaia, portugal	17
5	farnborough, hants, united kingdom	NA
6	santa monica, california, usa	61
7	washington, dc, usa	NA
8	timmins, ontario, canada	NA

user_id	location	age
9	germantown, tennessee, usa	NA
10	albacete, wisconsin, spain	26

These samples illustrate the original format of the books, ratings, and users tables before preprocessing. They provide transparency about the raw data sources used in the study.

6.2 Distribution Plots



These plots highlight the long-tailed distributions typical of recommendation datasets, motivating the filtering thresholds applied in preprocessing.

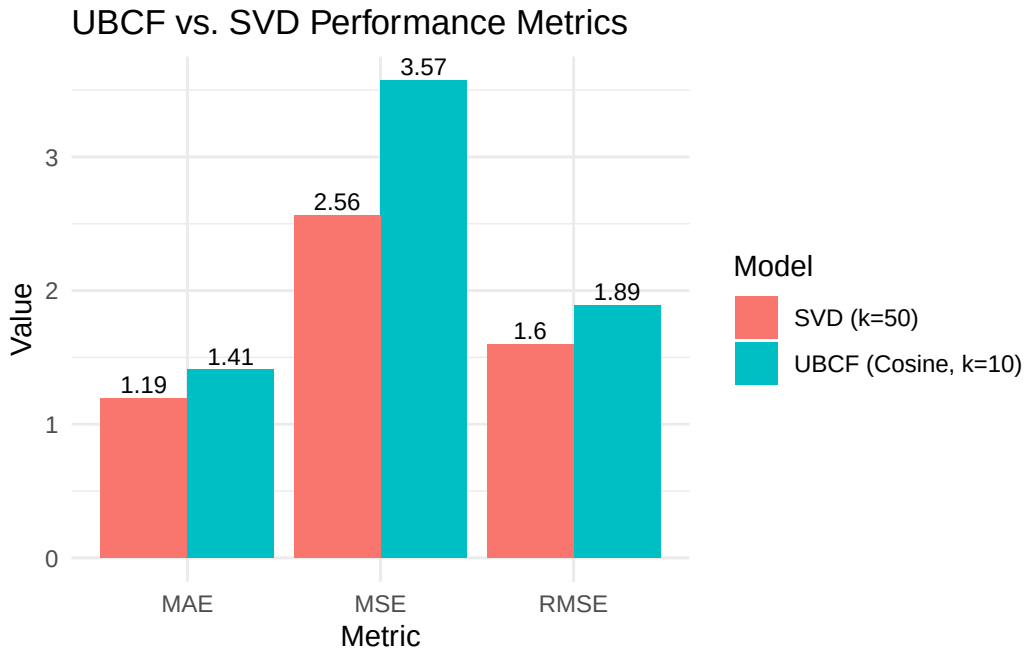
6.3 Cross-Validation Results Grid

Table 10: Cross-validation results for UBCF models

	Model	V1
RMSE...1	UBCF_cosine_k10	1.986
MSE...2	UBCF_cosine_k10	3.947
MAE...3	UBCF_cosine_k10	1.480
RMSE...4	UBCF_pearson_k30	2.038
MSE...5	UBCF_pearson_k30	4.160
MAE...6	UBCF_pearson_k30	1.552

This table provides the detailed RMSE and MAE values across different neighborhood sizes and similarity metrics, supporting the choice of UBCF with cosine similarity and $k = 10$.

6.4 Full Comparison of UBCF vs. SVD



This figure visually compares the predictive accuracy of UBCF and SVD across RMSE,

MSE, and MAE.

6.5 Shiny App Code Snippet

```
server <- function(input, output) {  
  model_reactive <- reactive({  
    Recommender(rating_matrix, method = "UBCF",  
                parameter = list(nn = input$k, method = "Cosine",  
                                normalize = "center"))  
  })  
  
  output$recommendations <- renderTable({  
    pred <- predict(model_reactive(), rating_matrix[input$user_id,],  
                   n = input$n)  
    as(pred, "list")  
  })  
}
```

This snippet illustrates how the UBCF model is embedded in the Shiny app, allowing interactive adjustment of neighborhood size and number of recommendations.

References

Aggarwal, C. C. (2016), *Recommender Systems: The Textbook*, Springer.

URL: <https://doi.org/10.1007/978-3-319-29659-3>

- APXML (2025), ‘Chapter 3: Neighborhood-Based Collaborative Filtering’, <https://apxml.com/courses/building-ml-recommendation-system/chapter-3-neighborhood-based-collaborative-filtering/making-predictions-weighted-averages>.
- Deb, D. (2025), ‘Choosing the right ‘k’ in knn: A practical guide’, <https://www.linkedin.com/pulse/choosing-right-k-knn-practical-guide-debasish-deb-1ydwf>. Accessed: December 14, 2025.
- for Managers, D. S. (2025), ‘The bias-variance trade-off’, https://dsm.business/08_knn/bias_variance.html. Accessed: December 14, 2025.
- GeeksforGeeks (2024a), ‘Curse of dimensionality in machine learning’, <https://www.geeksforgeeks.org/machine-learning/curse-of-dimensionality-in-machine-learning/>.
- GeeksforGeeks (2024b), ‘Why is knn a lazy learner?’, <https://www.geeksforgeeks.org/machine-learning/why-is-knn-a-lazy-learner/>.
- Ghosh, A. (2024), ‘Recommendation system: A complete guide’, <https://learnopencv.com/recommendation-system/#aioseo-types-of-recommendation-system>.
- Hahsler, M. (2022), *Introduction to Recommender Systems*, Data Science Series, Chapman and Hall/CRC.
- Hahsler, M. (2025), ‘recommenderlab documentation’, <https://cran.r-project.org/web/packages/recommenderlab/index.html>.
- Hssina, B., Grotta, A. & Erritali, M. (2021), ‘Recommendation system using the k-nearest neighbors and singular value decomposition algorithms’, *International Journal of Electrical and Computer Engineering* **11**(6), 5541–5548.
URL: <https://doi.org/10.11591/ijece.v11i6.pp5541-5548>

Kramer, O. (2013), *Dimensionality Reduction with Evolutionary Algorithms*, Springer.

URL: <https://doi.org/10.1007/978-3-642-38652-7>

Nikolakopoulos, A. N., Ning, X., Desrosiers, C. & Karypis, G. (2021), ‘Trust your neighbors: A comprehensive survey of neighborhood-based methods for recommender systems’, *arXiv preprint arXiv:2109.04584* .

URL: <https://arxiv.org/abs/2109.04584>

Ricci, F., Rokach, L. & Shapira, B., eds (2022), *Recommender Systems Handbook*, 3 edn, Springer.

URL: <https://doi.org/10.1007/978-1-0716-2197-4>

Sharma, S. (2020), ‘K-nearest neighbor’, <https://medium.com/swlh/k-nearest-neighbor-ca2593d7a3c4>.

Ziegler, C.-N. (2024), ‘Book-crossing dataset’, <https://www.kaggle.com/datasets/somnambwl/bookcrossing-dataset>.